GenAI analysis - Assignment 1

Gen-AI # 1 - Google Gemini

1. **I accessed this on my computer**
2. **My prompt:**

Write a C program that picks a secret number between 1 and 10.
The user gets up to 3 tries to guess the number. It must meet all of the following requirements:
• Use a fixed secret number (for example, 7).• Use printf to prompt the user and scanf to read their
guess.
• Use a while or for loop to let the user guess up to 3 times.
• Use if-else statements to check if the guess is too high,
too low, or correct.
• Stop the loop immediately if the user guesses the correct
number.
• Print a final win or lose message.

3. **The given code:**

```
#include <stdio.h>

int main() {
    int secret_number = 7;
    int guess;
    int win = 0;

    for (int attempt = 1; attempt <= 3; attempt++) {
        printf("Try %d/3 - Guess a number between 1 and 10: ", attempt);
        scanf("%d", &guess);

        if (guess == secret_number) {
          win = 1;
          break;
        } else if (guess > secret_number) {
            printf("Too high!\n");
        } else {
            printf("Too low!\n");
        }
    }
    if (win) {
       printf("Congratulations! You guessed the secret number!\n");
    } else {
       printf("Game over! You ran out of tries. The secret number was %d.\n", secret_number);
    }
    return 0;
```

}

Gen-AI # 2 Microsoft CoPilot

1. **Accessed CoPilot on my computer**
2. **Prompt:**

Write a C program that picks a secret number between 1 and 10.
The user gets up to 3 tries to guess the number. It must meet all of the following requirements:
• Use a fixed secret number (for example, 7).• Use printf to prompt the user and scanf to read their
guess.
• Use a while or for loop to let the user guess up to 3 times.
• Use if-else statements to check if the guess is too high,
too low, or correct.
• Stop the loop immediately if the user guesses the correct
number.
• Print a final win or lose message.

3. Given Code

```
#include <stdio.h>

int main() {

    int secret = 7;        // fixed secret number

    int guess;

    int tries = 0;

    while (tries < 3) {    // allow up to 3 guesses

        printf("Enter your guess (1–10): ");

        scanf("%d", &guess);

        if (guess == secret) {

            printf("Correct! You win!\n");

            return 0;      // stop immediately on correct guess

        } else if (guess < secret) {

            printf("Too low.\n");

        } else {
```

```
            printf("Too high.\n");
        }
        tries++;
    }
    printf("Sorry, you lose. The number was %d.\n", secret);
    return 0;
}
```

Code Comparison:

1. Correctness

The Gemini and CoPilot code have a good basic number guessing game. They set a secret number both to 7 and allow up to three guesses. They also both tell the user if the guess is too high or too low or correct. The gemini code uses a win variable and a break statement to stop the loop. Co Pilot immediately ends the program with return 0. Both programs have a weakness because they don't check whether or not scanf() receives an integer. They also don't stop someone from giving a number larger than 10 or less than 1.

2. Execution Time

Both programs have the same time complexity as O(1) with the max of three guesses. The gemini program uses a for loop that can run at most three times. CoPilot uses a while loop that also runs only three times. The iterations do a small number of comparisons and one input operation. The number of attempts is fixed so neither programs execution time grows with the size of input.

3. Space Complexity

Both programs have a space complexity of O(1) because of the fixed number of variables and do not store memory based on the number of guesses. The Gemini code has variables like secret_number, guess, win and attempt. CoPilot uses secret, guess, tries. Both programs use a constant amount of memory making them almost equal.

4. Maintainability

CoPilots code is slightly more maintainable in comparison to Gemini's code because it has a simpler control flow. When the user guesses correctly CoPilot immediately prints the winning message and exits the program using return 0. Gemini uses its win variable which is not necessary. CoPilot has an advantage in maintainability because its implementation is more together and concise with less unnecessary variables.

Selection:

I chose the Microsoft CoPilot program because it's simpler and easier to understand and maintain than the Gemini version. Both programs met the requirements and have similar time and space complexity. CoPilot just uses fewer variables and it ends the program easier than Gemini. I think CoPilot is a better choice to use for this assignment.

Code Improvement:

Correctness: I changed the while loop to a for loop to clearly limit the user to three attempts. The program correctly identifies guesses as correct, low, or high.

Execution: The execution time remains O(1) because the program performs at most three guesses. The program also immediately stops when the correct number is guessed

Space Complexity: I removed the separate tries variable and used the for loop's i variable instead. The space complexity remains O(1), but the program uses one fewer separate variable.

Maintainability: The for loop makes the three limit attempt easier to understand and modify. I also added a comment explaining each line of the code.